

Documentación del proyecto Trivia GenshikenCS.es



**Proyecto desarrollado durante las prácticas
Desarrollo de Aplicaciones Multiplataforma**

Alumno: Hamza El Khattabi
Pablo Alonso

Empresa: MARK & RAPH SL

Tutor de empresa: Rafael Cobos López

Curso: 2025/2026

Fecha: Junio de 2026

ÍNDICE

Documentación del proyecto Trivia GenshikenC.S.....	1
ÍNDICE.....	2
Introducción, objetivo y visión general del sistema.....	5
1. Introducción.....	5
2. Objetivo del proyecto.....	6
3. Contexto de las prácticas.....	6
4. Tecnologías utilizadas.....	7
4.1. Tecnologías de la aplicación Android.....	7
4.2. Tecnologías de la parte web.....	7
4.3. Base de datos.....	8
5. Estructura general del sistema.....	8
5.1. Aplicación Android.....	8
5.2. Panel de administración web.....	9
5.3. APIs PHP.....	9
5.4. Base de datos MySQL.....	10
6. Relación entre app, web, APIs y base de datos.....	10
7. Funcionamiento general del juego.....	10
8. Mejoras principales realizadas durante las prácticas.....	11
9. Resumen de la arquitectura.....	11
Parte web, panel de administración y APIs PHP.....	12
1. Introducción a la parte web.....	12
2. Panel de administración.....	13
3. Archivo config.php.....	13
4. Login y cierre de sesión del administrador.....	14
4.1. login.php.....	14
4.2. logout.php.....	14
5. Dashboard del panel.....	14
6. Gestión de preguntas.....	15
6.1. preguntas.php.....	15
6.2. crear_pregunta.php.....	16
6.3. editar_pregunta.php.....	17
6.4. eliminar_pregunta.php.....	17
6.5. ver_respuestas.php.....	17
7. Selector de imágenes.....	18
8. Gestión del ranking.....	18
8.1. ranking.php.....	18
8.2. reset_ranking.php.....	19
8.3. ganadores.php.....	19
9. Usuarios registrados.....	19
10. Instalaciones de la app.....	20

11. Gestión del gacha y espadas.....	20
11.1. espadas.php.....	21
11.2. crear_espada.php.....	21
11.3. editar_espada.php.....	22
11.4. eliminar_espada.php.....	22
12. Enlaces clicables en descripciones.....	23
13. Estilos del panel.....	23
14. APIs PHP.....	24
14.1. api/register.php.....	24
14.2. api/login.php.....	25
14.3. api/getPreguntas.php.....	25
14.4. api/guardarPuntuacion.php.....	26
14.5. api/getRanking.php.....	26
14.6. api/registrarDescarga.php.....	27
14.7. APIs de monedas y colección.....	27
15. Juego web de prueba.....	28
16. Imágenes del proyecto.....	28
17. Uso básico del panel de administración.....	29
18. Pruebas realizadas en la parte web.....	29
19. Mejoras realizadas en la web durante las prácticas.....	30
20. Conclusión de la parte web.....	30
Aplicación Android, base de datos, pruebas finales y conclusión.....	31
1. Introducción a la aplicación Android.....	31
2. Estructura general de la app Android.....	32
3. MainActivity.kt.....	33
4. Navegacion.kt.....	34
5. PantallaInicial.kt.....	34
6. Inicio de sesión y registro.....	35
6.1. InicioSesionCompetitivo.kt.....	35
6.2. InicioSesionCasual.kt.....	36
6.3. RegistroCompeti.kt.....	36
6.4. RegistroCasual.kt.....	37
7. UserSession.kt y GameSession.....	37
8. ApiService.kt.....	37
9. RetrofitClient.kt.....	38
10. UserRepository.kt.....	39
11. Juego.kt.....	39
12. Preguntas.kt.....	40
13. EstadoRespuesta.kt.....	40
14. Ranking.kt.....	41
15. Jugador.kt.....	41

16. Sistema de gacha.....	42
16.1. PantallaGacha.kt.....	42
16.2. EspadaOnline.kt.....	43
16.3. GachaState.kt.....	44
16.4. PantallaColeccion.kt.....	44
16.5. EspadaGacha.kt.....	44
17. Música y sonidos.....	45
17.1. MusicManager.kt.....	45
18. Tema visual y preferencias.....	45
18.1. ThemeState.kt.....	46
18.2. ThemePreferences.kt.....	46
19. Cambio de nombre.....	46
20. Términos de servicio.....	46
21. Curiosidades.....	46
22. Base de datos.....	47
22.1. Tabla usuarios.....	47
22.2. Tabla preguntas.....	48
22.3. Tabla respuestas.....	48
22.4. Tabla niveles.....	48
22.5. Tabla puntuaciones.....	48
22.6. Tabla ranking_mensual_historico.....	48
22.7. Tabla descargas.....	48
22.8. Tabla espadas.....	49
22.9. Tabla monedas_usuario.....	49
22.10. Tabla coleccion_usuario.....	49
23. Manual básico de uso de la app.....	49
23.1. Abrir la app.....	49
23.2. Registrarse.....	49
23.3. Iniciar sesión en competitivo.....	50
23.4. Jugar.....	50
23.5. Consultar ranking.....	50
23.6. Usar el gacha.....	50
23.7. Ver colección.....	50
24. Manual básico de uso del panel.....	50
25. Pruebas realizadas.....	51
25.1. Pruebas de Android.....	51
25.2. Pruebas de web.....	51
25.3. Pruebas de API.....	52
26. Mejoras realizadas en la app durante las prácticas.....	52
27. Mejoras realizadas en la base de datos.....	52
28. Posibles mejoras futuras.....	53
29. Conclusión final.....	53

Introducción, objetivo y visión general del sistema

1. Introducción

Durante el periodo de prácticas se ha trabajado en el desarrollo y mejora del proyecto **Trivia GenshikenC.S.**, una aplicación tipo trivia centrada principalmente en el mundo del anime, las espadas y la cultura relacionada con Genshiken. El proyecto está formado por dos partes principales: una **aplicación Android**, que es la parte que utiliza el usuario final, y una **parte web/backend**, que permite administrar el contenido desde un panel de administración y conectar la app con una base de datos.

La finalidad del proyecto es ofrecer una experiencia de juego sencilla, visual y entretenida, donde los usuarios puedan registrarse, iniciar sesión, responder preguntas por niveles, conseguir puntuaciones, aparecer en un ranking y desbloquear espadas mediante un sistema de gacha. Además, desde el panel web se puede gestionar el contenido de forma dinámica, sin tener que modificar directamente el código de la aplicación.

El sistema se ha ido ampliando durante las prácticas con nuevas funcionalidades, mejoras visuales, integración con base de datos, conexión entre Android y PHP, gestión de imágenes, ranking, control de usuarios, registro de instalaciones, música en la app y gestión del gacha desde el panel de administración.

2. Objetivo del proyecto

El objetivo principal del proyecto es crear una aplicación de trivia funcional, conectada a un backend real, que permita a los usuarios jugar desde Android y que al mismo tiempo pueda ser administrada fácilmente desde una web.

Los objetivos principales son:

- Permitir el registro e inicio de sesión de usuarios.
- Diferenciar entre modo casual y modo competitivo.
- Cargar preguntas desde una base de datos.
- Organizar las preguntas por niveles.
- Mostrar imágenes asociadas a preguntas y respuestas.
- Guardar puntuaciones de los usuarios.
- Mostrar un ranking de mejores jugadores.
- Registrar instalaciones o usos de la app desde el modo competitivo.
- Crear un sistema de gacha con espadas desbloqueables.
- Mostrar una colección de espadas obtenidas por el usuario.
- Permitir gestionar preguntas, respuestas y espadas desde un panel de administración.
- Facilitar la subida de imágenes desde el propio panel web.
- Añadir música de fondo al menú y al juego.
- Permitir enlaces clicables en las descripciones de las espadas.

El resultado final es una aplicación Android conectada con una web administrativa y una base de datos MySQL, preparada para poder añadir o modificar contenido sin recompilar constantemente la app.

3. Contexto de las prácticas

El proyecto se ha desarrollado dentro del periodo de prácticas en empresa, con el objetivo de aplicar conocimientos adquiridos durante el ciclo de Desarrollo de Aplicaciones Multiplataforma.

Durante las prácticas se ha trabajado tanto en la parte de desarrollo web como en la parte de integración con Android. La parte web ha servido como panel de administración y backend, mientras que la app Android funciona como producto final para el usuario.

El trabajo realizado ha incluido tareas de programación, pruebas, conexión con hosting, gestión de base de datos, subida de archivos por FTP, organización del proyecto, corrección de errores y preparación del sistema para su demostración final.

También se han realizado pruebas tanto en local como en hosting, comprobando que las distintas partes del sistema funcionen correctamente: panel de administración, preguntas, ranking, usuarios registrados, instalaciones, gacha, colección, música y enlaces.

4. Tecnologías utilizadas

Para el desarrollo del proyecto se han utilizado varias tecnologías, separadas principalmente entre la parte Android, la parte web y la base de datos.

4.1. Tecnologías de la aplicación Android

La app Android está desarrollada principalmente con:

- **Kotlin**, como lenguaje principal de programación.
- **Jetpack Compose**, para crear la interfaz gráfica de la aplicación.
- **Android Studio**, como entorno de desarrollo.
- **Retrofit**, para realizar llamadas HTTP a las APIs del backend.
- **Gson Converter**, para convertir las respuestas JSON de la API en objetos Kotlin.
- **Coil**, para cargar imágenes desde URLs en la app.
- **MediaPlayer**, para reproducir música de fondo en el menú y en la partida.
- **Navigation Compose**, para gestionar la navegación entre pantallas.
- **Material 3**, para componentes visuales como botones, tarjetas, textos y formularios.

La app se conecta al backend mediante una URL base configurada en `RetrofitClient.kt`, y las peticiones disponibles se definen en `ApiService.kt`.

4.2. Tecnologías de la parte web

La parte web y el panel de administración están desarrollados con:

- **PHP**, para la lógica del servidor.
- **MySQL**, como sistema de base de datos.
- **HTML**, para la estructura de las páginas.
- **CSS**, para el diseño visual del panel y la web.
- **JavaScript**, para algunas funciones del juego web y ranking.
- **phpMyAdmin**, para gestionar la base de datos.
- **XAMPP**, para pruebas locales.
- **FileZilla**, para subir archivos al hosting.
- **Hosting web**, donde está publicada la web y las APIs.

El panel de administración se encuentra dentro de la carpeta admin, mientras que las APIs se organizan en carpetas como api y api_genshiken.

4.3. Base de datos

La base de datos utilizada es MySQL. En ella se almacenan usuarios, preguntas, respuestas, niveles, puntuaciones, ranking histórico, instalaciones, monedas, espadas y colecciones de usuario.

Algunas tablas importantes son:

- usuarios
- preguntas
- respuestas
- niveles
- puntuaciones
- ranking_mensual_historico
- descargas
- espadas
- monedas_usuario
- coleccion_usuario

La base de datos permite que tanto el panel web como la app Android trabajen sobre la misma información.

5. Estructura general del sistema

El sistema está dividido en cuatro bloques principales:

1. **Aplicación Android**
2. **Panel de administración web**
3. **APIs PHP**
4. **Base de datos MySQL**

Cada bloque cumple una función diferente, pero todos están conectados entre sí.

5.1. Aplicación Android

La aplicación Android es la parte que utiliza el jugador. Desde ella se puede acceder a la pantalla inicial, iniciar sesión, registrarse, jugar, consultar el ranking, usar el gacha y ver la colección de espadas.

Las pantallas principales de la app son:

- Pantalla inicial.
- Inicio de sesión casual.
- Inicio de sesión competitivo.
- Registro casual.
- Registro competitivo.
- Pantalla de juego.
- Ranking.
- Gacha.
- Colección de espadas.
- Cambio de nombre.
- Términos de servicio.

La app consume información desde el backend, como preguntas, respuestas, espadas, monedas, ranking y usuarios.

5.2. Panel de administración web

El panel de administración permite gestionar el contenido del juego desde el navegador. Es una parte importante porque evita tener que modificar manualmente la base de datos para añadir preguntas o espadas.

Desde el panel se puede:

- Iniciar sesión como administrador.
- Ver estadísticas generales en el dashboard.
- Gestionar preguntas.
- Crear preguntas con imágenes.
- Editar preguntas.
- Eliminar preguntas.
- Ver respuestas.
- Gestionar el ranking.
- Ver usuarios registrados.
- Ver instalaciones de la app.
- Gestionar espadas del gacha.
- Crear, editar y eliminar espadas.
- Añadir enlaces en las descripciones de espadas.

El panel se encuentra en la carpeta admin.

5.3. APIs PHP

Las APIs PHP son el puente entre la app Android, la web y la base de datos. La app no se conecta directamente a MySQL, sino que llama a archivos PHP que reciben datos, consultan la base de datos y devuelven respuestas.

Algunas APIs importantes son:

- Registro de usuarios.
- Inicio de sesión.
- Obtención de preguntas.
- Guardado de puntuaciones.
- Obtención de ranking.
- Guardado y obtención de monedas.
- Guardado y obtención de espadas desbloqueadas.
- Obtención de espadas del gacha.
- Registro de instalaciones de la app.

Gracias a estas APIs, la aplicación Android puede funcionar conectada al hosting y recibir datos actualizados.

5.4. Base de datos MySQL

La base de datos es el centro del proyecto. Guarda toda la información que necesitan tanto la app como el panel de administración.

Por ejemplo:

- Cuando un usuario se registra, se guarda en la tabla usuarios.
- Cuando el administrador crea una pregunta, se guarda en preguntas.
- Las respuestas se guardan en respuestas.
- Las puntuaciones se guardan en puntuaciones.
- Las instalaciones se guardan en descargas.
- Las espadas del gacha se guardan en espadas.
- Las monedas de cada usuario se guardan en monedas_usuario.
- Las espadas desbloqueadas por cada usuario se guardan en coleccion_usuario.

Esto permite mantener la información organizada y reutilizable.

6. Relación entre app, web, APIs y base de datos

El funcionamiento general del sistema es el siguiente:

1. El administrador entra al panel web.
2. Desde el panel crea o modifica preguntas, respuestas, imágenes y espadas.
3. Esa información se guarda en la base de datos MySQL.
4. La app Android realiza llamadas a las APIs PHP.
5. Las APIs consultan la base de datos.
6. La app recibe los datos y los muestra al usuario.
7. Cuando el usuario juega, la app guarda puntuaciones, monedas, espadas desbloqueadas e instalaciones mediante APIs.
8. El panel permite revisar posteriormente esos datos.

De esta forma, el proyecto queda conectado de forma completa:

- La **web** administra.
- Las **APIs** comunican.
- La **base de datos** almacena.
- La **app Android** muestra y permite jugar.

7. Funcionamiento general del juego

El usuario entra en la app y encuentra una pantalla inicial con el logo, música de fondo y accesos a las distintas secciones. Desde ahí puede elegir entre modo casual o modo competitivo.

El modo competitivo es el más importante para el funcionamiento completo del sistema, porque permite iniciar sesión, jugar, guardar puntuaciones, aparecer en el ranking y registrar la instalación de la app.

Durante la partida, la app muestra preguntas con imagen y varias respuestas posibles. El usuario selecciona una opción y el sistema indica si ha acertado o fallado. Al terminar, se calcula la puntuación y se guarda en la base de datos.

Además del juego de preguntas, la app incluye un sistema de gacha. El usuario puede conseguir monedas jugando y usarlas para obtener espadas aleatorias. Las espadas desbloqueadas se muestran en la colección, junto con su nombre, rareza, descripción, imagen y enlaces clicables si la descripción contiene una URL.

8. Mejoras principales realizadas durante las prácticas

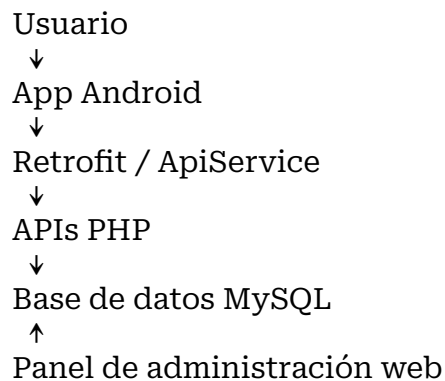
Durante el desarrollo de las prácticas se han añadido y mejorado varias partes importantes del proyecto:

- Mejora del panel de administración.
- Gestión completa de preguntas y respuestas.
- Selector de imágenes para preguntas y respuestas.
- Gestión del ranking.
- Registro de usuarios.
- Apartado de instalaciones de la app.
- Gestión de espadas del gacha desde el panel.
- Subida de imágenes para espadas.
- Enlaces clicables en las descripciones de espadas.
- Conexión entre la app Android y el backend.
- Sistema de monedas.
- Sistema de colección de espadas.
- Música de menú y música de juego.
- Registro de instalación desde el inicio de sesión competitivo.
- Añadido de preguntas para el nivel 2.

Estas mejoras han permitido que el proyecto pase de ser una demo básica a un sistema más completo, conectado y fácil de administrar.

9. Resumen de la arquitectura

La arquitectura del proyecto puede resumirse así:



El usuario juega desde Android.
El administrador gestiona desde la web.
Las APIs conectan ambas partes con la base de datos.
La base de datos mantiene toda la información del sistema.

Esta separación permite que el proyecto sea más organizado, fácil de mantener y ampliable en el futuro.

Parte web, panel de administración y APIs PHP

1. Introducción a la parte web

La parte web del proyecto **Trivia GenshikenC.S.** cumple dos funciones principales. Por un lado, sirve como página web pública sencilla del proyecto, donde se puede acceder al juego web de prueba y al ranking. Por otro lado, incluye un **panel de administración** desde el que se puede gestionar todo el contenido importante que después utiliza la app Android.

La web está desarrollada principalmente con **PHP, HTML, CSS, JavaScript y MySQL**. La base de datos se gestiona mediante phpMyAdmin, y la conexión entre Android y la base de datos se realiza mediante archivos PHP que funcionan como APIs.

La parte web se encuentra dentro de la carpeta principal:

WEB_genshi/

Dentro de esta carpeta se organizan varias secciones:

```
admin/  
api/  
api_genshiken/  
css/  
js/  
img/  
index.html  
juego.html  
ranking.html
```

La parte más importante para la administración del proyecto es la carpeta:

```
WEB_genshi/admin/
```

2. Panel de administración

El panel de administración permite controlar el contenido del juego sin tener que modificar directamente la base de datos. Esto hace que el proyecto sea mucho más cómodo de mantener, ya que desde el navegador se pueden crear preguntas, editar respuestas, revisar usuarios, ver instalaciones, gestionar el ranking y administrar las espadas del gacha.

El acceso al panel se realiza desde:

```
http://www.shopkatanas.com/WEB_genshi/admin/login.php
```

El panel está protegido mediante sesión de administrador. Si un usuario intenta acceder a una página interna sin haber iniciado sesión, el sistema lo redirige automáticamente al login.

3. Archivo config.php

El archivo:

```
admin/config.php
```

es uno de los archivos más importantes del panel.

Su función principal es centralizar la configuración general del panel. En este archivo se realiza:

- El inicio de sesión PHP mediante `session_start()`.
- La definición del usuario y contraseña del administrador.
- La configuración de conexión con la base de datos.
- La función `conectarDB()`, que devuelve una conexión MySQL preparada para trabajar con UTF-8.

Este archivo evita repetir la conexión a la base de datos en cada página. Las demás páginas del panel simplemente incluyen:

```
require_once "config.php";
```

y después llaman a:

```
$conexion = conectarDB();
```

De esta manera, si cambia la configuración de la base de datos, solo hay que modificar config.php.

4. Login y cierre de sesión del administrador

4.1. login.php

El archivo:

admin/login.php

permite acceder al panel de administración.

El formulario pide:

- Usuario.
- Contraseña.

Cuando el usuario introduce los datos, el archivo comprueba si coinciden con las credenciales definidas en config.php. Si son correctas, se guarda en sesión que el administrador está logueado y se redirige al dashboard.

La sesión se guarda con variables como:

```
$_SESSION["admin_logueado"]  
$_SESSION["admin_usuario"]
```

Si las credenciales no son correctas, se muestra un mensaje de error.

4.2. logout.php

El archivo:

admin/logout.php

sirve para cerrar la sesión del administrador.

Su función es destruir la sesión actual y redirigir al login. Esto permite salir del panel de forma segura.

5. Dashboard del panel

El archivo:

`admin/dashboard.php`

muestra la pantalla principal del panel de administración.

Desde esta pantalla se ven estadísticas generales del sistema, como:

- Total de preguntas.
- Total de respuestas.
- Total de niveles.
- Total de instalaciones detectadas.
- Total de espadas del gacha.

Además, desde el dashboard se accede a las principales secciones del panel:

- Preguntas y respuestas.
- Ranking.
- Usuarios registrados.
- Instalaciones de la app.
- Gacha / espadas.

Una mejora importante de este archivo es que comprueba si existen las tablas antes de intentar contar registros. Esto evita errores si alguna tabla no existe todavía o si se trabaja en un entorno local diferente al hosting.

6. Gestión de preguntas

La gestión de preguntas es una de las partes principales del panel. Permite crear, editar, visualizar y eliminar preguntas del juego.

Las preguntas se guardan en la tabla:

`preguntas`

y sus respuestas se guardan en la tabla:

`respuestas`

Cada pregunta pertenece a un nivel mediante el campo:

`nivel_id`

6.1. preguntas.php

El archivo:

admin/preguntas.php

muestra el listado de preguntas del juego.

Desde esta página el administrador puede:

- Ver las preguntas existentes.
- Buscar preguntas.
- Ver la imagen asociada a cada pregunta.
- Ver a qué nivel pertenece cada pregunta.
- Acceder a la edición de una pregunta.
- Eliminar preguntas.
- Ver las respuestas asociadas.

Este archivo está preparado para funcionar con diferentes versiones de la tabla preguntas. Por ejemplo, puede detectar si la tabla usa columnas llamadas:

pregunta / imagen

o:

pregunta_texto / pregunta_imagen

Esto hace que el panel sea más flexible y evita errores cuando la estructura cambia entre local y hosting.

6.2. crear_pregunta.php

El archivo:

admin/crear_pregunta.php

permite crear una nueva pregunta desde el panel.

El formulario recoge:

- Texto de la pregunta.
- Imagen de la pregunta.
- Nivel al que pertenece.
- Cuatro respuestas.
- Imagen opcional para cada respuesta.
- Respuesta correcta.

Una de las mejoras importantes realizadas durante las prácticas fue añadir un **selector de archivos** para las imágenes. Antes había que escribir manualmente la ruta de la imagen, lo cual era menos cómodo y más propenso a errores.

Ahora el administrador puede seleccionar la imagen directamente desde el ordenador. El sistema sube automáticamente la imagen a una carpeta según el nivel, por ejemplo:

img/nivel1/
img/nivel2/

y guarda en la base de datos una ruta como:

nivel2/nombre_imagen.jpg

Esto facilita mucho la creación de preguntas nuevas.

6.3. editar_pregunta.php

El archivo:

admin/editar_pregunta.php

permite modificar una pregunta ya existente.

Desde esta pantalla se puede cambiar:

- Texto de la pregunta.
- Imagen de la pregunta.
- Nivel.
- Texto de las respuestas.
- Imágenes de las respuestas.
- Respuesta correcta.

También incluye selector de archivos. Si el administrador no selecciona una imagen nueva, el sistema conserva la imagen anterior.

Esto es importante porque permite editar solo una parte de la pregunta sin perder los datos que ya estaban bien.

6.4. eliminar_pregunta.php

El archivo:

admin/eliminar_pregunta.php

permite eliminar una pregunta.

Su función es borrar la pregunta seleccionada de la base de datos. Normalmente también se eliminan o quedan asociadas las respuestas según cómo esté estructurada la base de datos y las relaciones.

Esta opción debe usarse con cuidado, ya que elimina contenido del juego.

6.5. ver_respuestas.php

El archivo:

admin/ver_respuestas.php

permite ver las respuestas asociadas a una pregunta concreta.

Sirve para revisar rápidamente si una pregunta tiene:

- Cuatro respuestas.
- Una respuesta correcta.
- Texto correcto.
- Imágenes si las hay.

Es útil para comprobar el contenido después de crear o editar preguntas.

7. Selector de imágenes

Una mejora importante del panel fue añadir selectores de archivos para trabajar con imágenes.

Antes, para añadir una imagen a una pregunta, había que escribir una ruta manualmente, por ejemplo:

nivel1/q1.png

Esto podía provocar errores si se escribía mal la ruta o si el archivo no estaba en la carpeta correcta.

Con la mejora realizada, el administrador puede elegir la imagen desde el ordenador y el sistema se encarga de:

1. Comprobar que se ha subido correctamente.
2. Validar la extensión del archivo.
3. Crear la carpeta del nivel si no existe.
4. Mover la imagen a la carpeta correspondiente.
5. Guardar en la base de datos la ruta correcta.

Esto se aplica tanto a imágenes de preguntas como a imágenes de respuestas.

8. Gestión del ranking

El ranking permite mostrar las mejores puntuaciones de los usuarios.

Los archivos principales son:

admin/ranking.php
admin/reset_ranking.php
admin/ganadores.php

8.1. ranking.php

El archivo:

admin/ranking.php

muestra las puntuaciones de los jugadores.

El ranking se basa en la tabla:

puntuaciones

En esta tabla se guardan datos como:

- Usuario.
- Nombre.
- Puntos.
- Tiempo.
- Fecha.

El ranking permite comprobar qué jugadores han conseguido mejores resultados en el juego.

8.2. reset_ranking.php

El archivo:

admin/reset_ranking.php

sirve para reiniciar el ranking.

Antes de borrar o reiniciar el ranking actual, el sistema puede guardar la información en una tabla de histórico mensual:

ranking_mensual_historico

Esto permite conservar un registro de rankings anteriores y no perder completamente los resultados.

8.3. ganadores.php

El archivo:

admin/ganadores.php

está relacionado con la visualización o gestión de ganadores mensuales.

Esta parte sirve para consultar los resultados históricos y mantener un control de los mejores jugadores por periodo.

9. Usuarios registrados

El archivo:

admin/usuarios.php

muestra los usuarios registrados en la app.

Estos usuarios se guardan en la tabla:

usuarios

Desde esta sección se puede revisar información como:

- Nombre de usuario.
- Correo electrónico.
- Estado de verificación.
- Fecha de registro.

Esta sección es diferente de instalaciones. Un usuario registrado representa una cuenta creada en el sistema. En cambio, una instalación representa un uso detectado de la app desde un dispositivo.

10. Instalaciones de la app

El archivo:

admin/descargas.php

muestra las instalaciones o primeros usos detectados desde la app.

La información se guarda en la tabla:

descargas

Esta tabla permite guardar datos como:

- Usuario asociado.
- Nombre de usuario.
- Dispositivo.
- Versión de la app.
- Fecha de registro.

Aunque el archivo se llame registrarDescarga.php, realmente se usa para registrar instalaciones o primeros usos de la app.

Durante las prácticas se corrigió esta parte para que, al iniciar sesión en **modo competitivo**, la app pudiera registrar una instalación en el panel.

También se evitó que se generen duplicados. Es decir, si el mismo usuario inicia sesión varias veces desde el mismo dispositivo y con la misma versión de la app, no se registra como instalación nueva repetida.

Esto hace que el panel sea más limpio y que el contador de instalaciones sea más realista.

11. Gestión del gacha y espadas

Una de las mejoras importantes del proyecto fue añadir una sección específica para gestionar las espadas del gacha desde el panel de administración.

Los archivos principales son:

```
admin/espadas.php
admin/crear_espada.php
admin/editar_espada.php
admin/eliminar_espada.php
```

Las espadas se guardan en la tabla:

espadas

y las imágenes se almacenan en:

img/gacha/

11.1. espadas.php

El archivo:

admin/espadas.php

muestra el listado de espadas del gacha.

Desde esta página se puede ver:

- ID de la espada.
- Imagen.
- Nombre.
- Rareza.
- Descripción.
- URL de la imagen.
- Acciones de editar o eliminar.

También se añadió una mejora para que los enlaces escritos dentro de la descripción aparezcan como enlaces clicables. Por ejemplo, si la descripción contiene:

<http://www.google.com>

el panel lo muestra en azul, subrayado y permite abrirlo en otra pestaña.

Esto es útil para añadir enlaces a ofertas, páginas externas o información relacionada con una espada.

11.2. crear_espada.php

El archivo:

admin/crear_espada.php

permite crear una nueva espada para el gacha.

El formulario recoge:

- Nombre de la espada.
- Rareza.
- Descripción.
- Imagen.

La imagen se selecciona desde el ordenador y se sube automáticamente a:

img/gacha/

En la base de datos se guarda la URL completa de la imagen en el campo:

imagen_url

Las rarezas utilizadas son:

COMUN
RARA
EPICA
LEGENDARIA

Esto permite ampliar el catálogo del gacha sin tocar el código Android directamente, siempre que la app consuma las espadas desde la base de datos.

11.3. editar_espada.php

El archivo:

admin/editar_espada.php

permite modificar una espada existente.

Desde esta pantalla se puede cambiar:

- Nombre.
- Rareza.
- Descripción.
- Imagen.

Si no se selecciona una imagen nueva, se conserva la imagen anterior.

Esto permite corregir textos, añadir enlaces a la descripción o actualizar imágenes sin tener que borrar la espada y crearla de nuevo.

11.4. eliminar_espada.php

El archivo:

admin/eliminar_espada.php

permite eliminar una espada del catálogo del gacha.

Esta acción elimina la espada de la tabla espadas. Se debe usar con cuidado, especialmente si esa espada ya está asociada a usuarios en la colección.

12. Enlaces clicables en descripciones

Una mejora añadida al panel fue la detección automática de enlaces dentro de las descripciones de las espadas.

La función detecta enlaces que empiezan por:

http://
https://

y los convierte en enlaces HTML clicables.

Esto permite que el administrador pueda escribir una descripción como:

Espada disponible con oferta en <http://www.google.com>

y que en el panel se muestre el enlace de forma interactiva.

Además, como la descripción se guarda en base de datos, la app Android también puede recibir ese texto y mostrarlo. En la app se realizó la mejora correspondiente para que el enlace también pudiera pulsarse desde la colección.

13. Estilos del panel

El archivo:

admin/style.css

contiene los estilos principales del panel de administración.

Define el aspecto de:

- Login.
- Dashboard.
- Tarjetas del panel.
- Botones.
- Tablas.
- Formularios.
- Mensajes de error o éxito.

Durante el desarrollo se ajustaron estilos para que los botones fueran más claros, clicables y visualmente coherentes con el resto del panel.

14. APIs PHP

Además del panel visual, la web incluye varias APIs PHP que permiten conectar la app Android con la base de datos.

Las APIs principales se encuentran en:

WEB_genshi/api/

y también existe una carpeta:

WEB_genshi/api_genshiken/

usada por parte de la app Android.

Las APIs reciben datos mediante POST o GET, consultan o modifican la base de datos y devuelven respuestas en texto o JSON.

14.1. api/register.php

El archivo:

api/register.php

permite registrar usuarios desde la app Android.

Recibe:

- Nombre.
- Correo.
- Contraseña.

Guarda el usuario en la tabla usuarios y devuelve respuestas simples como:

OK
EXISTE
ERROR

Esto permite que la app sepa si el registro se hizo correctamente, si el correo ya existe o si hubo algún error.

14.2. api/login.php

El archivo:

api/login.php

permite iniciar sesión desde Android.

Recibe:

- Correo.
- Contraseña.

Busca el usuario en la tabla usuarios y devuelve un JSON con datos como:

```
{  
  "status": "OK",  
  "id": 1,  
  "nombre": "Usuario"  
}
```

La app utiliza esta respuesta para guardar el usuario en sesión y permitirle acceder al juego.

14.3. api/getPreguntas.php

El archivo:

api/getPreguntas.php

devuelve preguntas de un nivel concreto en formato JSON.

La app o el juego web pueden solicitar preguntas indicando el nivel. Por ejemplo:

nivel=1

nivel=2

La API consulta la tabla preguntas y también recupera las respuestas asociadas desde la tabla respuestas.

Esta API permite que el contenido del juego dependa de la base de datos y no esté completamente fijo en el código.

14.4. api/guardarPuntuacion.php

El archivo:

api/guardarPuntuacion.php

recibe la puntuación de un jugador y la guarda en la tabla:

puntuaciones

Puede recibir datos como:

- Nombre.
- Puntos.
- Tiempo.
- Usuario asociado.

Esto permite guardar los resultados de las partidas y después mostrarlos en el ranking.

14.5. api/getRanking.php

El archivo:

api/getRanking.php

devuelve el ranking de mejores jugadores.

Ordena principalmente por:

- Más puntos.
- Menor tiempo.
- Fecha.

La respuesta se devuelve en formato JSON para que pueda ser usada por la app o la web.

14.6. api/registrarDescarga.php

El archivo:

api/registrarDescarga.php

registra instalaciones o primeros usos de la app.

Recibe datos como:

- usuario_id
- nombre_usuario
- dispositivo
- version_app

Después guarda esa información en la tabla:

descargas

Durante las pruebas se comprobó mediante PowerShell que la API respondía correctamente con:

Instalación registrada correctamente.

Esto permitió confirmar que el hosting y la base de datos estaban funcionando.

14.7. APIs de monedas y colección

También existen APIs relacionadas con el sistema de gacha:

```
api/guardar_monedas.php  
api/guardar_espada.php  
api/obtener_espadas.php
```

Estas APIs permiten:

- Guardar monedas del usuario.
- Guardar espadas desbloqueadas.
- Obtener las espadas que ya tiene un usuario.

Están relacionadas con las tablas:

```
monedas_usuario  
coleccion_usuario
```

15. Juego web de prueba

Además de la app Android, el proyecto incluye una versión web sencilla del juego.

Los archivos principales son:

```
index.html  
juego.html  
ranking.html  
js/juego.js  
js/ranking.js  
css/juego.css
```

Esta parte funciona como demo o prueba del sistema. Permite comprobar preguntas, puntuaciones y ranking desde el navegador.

Aunque el producto final principal es la app Android, el juego web ayuda a validar que las APIs y la base de datos funcionan correctamente.

16. Imágenes del proyecto

Las imágenes se organizan dentro de la carpeta:

img/

Algunas subcarpetas importantes son:

img/nivel1/

img/nivel2/

img/gacha/

Las imágenes de preguntas se guardan por nivel:

img/nivel1/

img/nivel2/

Las imágenes de espadas del gacha se guardan en:

img/gacha/

Esta separación ayuda a mantener el proyecto ordenado y evita mezclar imágenes de preguntas con imágenes del gacha.

17. Uso básico del panel de administración

Para usar el panel de administración, el flujo básico es:

1. Entrar en el login del panel.
2. Introducir usuario y contraseña de administrador.
3. Acceder al dashboard.
4. Elegir la sección que se quiere gestionar.
5. Crear, editar o revisar contenido.
6. Guardar cambios.
7. Comprobar desde la app o desde la web que los datos aparecen correctamente.

Por ejemplo, para crear una pregunta:

1. Entrar en el panel.
2. Ir a preguntas.
3. Pulsar en crear pregunta.
4. Escribir el texto.
5. Seleccionar nivel.
6. Subir imagen.
7. Escribir las respuestas.
8. Marcar la correcta.
9. Guardar.
10. Probar en la app.

Para crear una espada:

1. Entrar en el panel.
2. Ir a Gacha / Espadas.
3. Pulsar en nueva espada.
4. Escribir nombre, rareza y descripción.
5. Subir imagen.
6. Guardar.
7. Comprobar que aparece en el listado y en la app.

18. Pruebas realizadas en la parte web

Durante las prácticas se realizaron varias pruebas para comprobar el funcionamiento del panel y las APIs:

- Inicio de sesión en el panel.
- Acceso al dashboard.
- Creación de preguntas.
- Edición de preguntas.
- Subida de imágenes desde el panel.
- Visualización de preguntas creadas.
- Creación del nivel 2 con nuevas preguntas.
- Consulta del ranking.
- Revisión de usuarios registrados.
- Registro de instalaciones desde la app.
- Prueba de registrarDescarga.php mediante PowerShell.
- Creación de espadas del gacha.
- Edición de espadas.
- Eliminación de espadas de prueba.
- Visualización de enlaces clicables en descripciones.
- Comprobación de imágenes del gacha.
- Prueba de conexión entre app Android y hosting.

Estas pruebas permitieron confirmar que el panel web y el backend funcionaban correctamente.

19. Mejoras realizadas en la web durante las prácticas

Las principales mejoras realizadas en la parte web fueron:

- Mejora del panel de administración.
- Creación de nuevas secciones del dashboard.
- Gestión más completa de preguntas y respuestas.
- Selector de imágenes para preguntas.
- Selector de imágenes para respuestas.
- Corrección de rutas de imágenes.
- Adaptación para funcionar en local y hosting.
- Sección de usuarios registrados.
- Sección de instalaciones de la app.
- API para registrar instalaciones.
- Gestión completa de espadas del gacha.
- Subida de imágenes de espadas.

- Edición de rareza, descripción e imagen de espadas.
- Enlaces clicables en descripciones de espadas.
- Limpieza visual del panel.
- Pruebas de APIs desde navegador, app y PowerShell.

Estas mejoras hicieron que la parte web pasara de ser un panel básico a una herramienta de administración mucho más completa.

20. Conclusión de la parte web

La parte web del proyecto es fundamental porque permite administrar el contenido del juego y conectar la app Android con la base de datos.

Gracias al panel de administración, el proyecto no depende de modificar código cada vez que se quiere añadir una pregunta, cambiar una imagen o crear una nueva espada. El administrador puede hacerlo directamente desde el navegador.

Además, las APIs PHP permiten que la app Android reciba preguntas, guarde puntuaciones, gestione el gacha, registre instalaciones y consulte información actualizada.

En resumen, la parte web cumple tres funciones principales:

Administrar contenido
Conectar la app con la base de datos
Controlar usuarios, ranking, instalaciones y gacha

Esto convierte al proyecto en un sistema completo, donde la app Android y el panel web trabajan juntos sobre una misma base de datos.

Aplicación Android, base de datos, pruebas finales y conclusión

1. Introducción a la aplicación Android

La aplicación Android es la parte principal del proyecto de cara al usuario final. Es la aplicación que permite jugar al trivia, iniciar sesión, registrarse, consultar el ranking, usar el sistema de gacha y ver la colección de espadas desbloqueadas.

La app está desarrollada con **Kotlin** y **Jetpack Compose**, utilizando una estructura basada en pantallas y clases auxiliares. La comunicación con la base de datos no se realiza directamente desde Android, sino mediante llamadas HTTP a las APIs PHP del backend.

La app se conecta al hosting mediante Retrofit y consume datos como:

- Usuarios.
- Preguntas.
- Respuestas.
- Ranking.
- Puntuaciones.
- Monedas.
- Espadas del gacha.
- Espadas desbloqueadas por el usuario.
- Registro de instalaciones.

La aplicación utiliza una navegación por pantallas, gestionada desde Navegacion.kt, y mantiene algunos datos temporales de sesión en objetos como UserSession, GameSession y GachaState.

2. Estructura general de la app Android

La parte Android se encuentra principalmente dentro de la ruta:

app/src/main/java/com/example/proyecto_genshiken/

Dentro de esta carpeta se encuentran las clases principales del proyecto, entre ellas:

MainActivity.kt

Navegacion.kt

PantallaInicial.kt

InicioSesionCompetitivo.kt

InicioSesionCasual.kt

RegistroCompeti.kt

RegistroCasual.kt

Juego.kt

Ranking.kt

PantallaGacha.kt

PantallaColeccion.kt

ApiService.kt

RetrofitClient.kt

UserRepository.kt

UserSession.kt

GachaState.kt

EspadaOnline.kt

EspadaGacha.kt

Preguntas.kt

Jugador.kt

MusicManager.kt

ThemeState.kt

ThemePreferences.kt

CambiarNombre.kt

TerminosServicios.kt

También se utilizan recursos gráficos y de sonido dentro de:

app/src/main/res/

Algunas carpetas importantes de recursos son:

drawable/

raw/

values/

En drawable se guardan imágenes como logos y espadas.

En raw se guardan sonidos y música.

En values se guardan recursos como colores, temas y textos.

3. MainActivity.kt

MainActivity.kt es el punto de entrada principal de la aplicación Android.

Su función es iniciar la app y cargar la interfaz principal usando Jetpack Compose. Dentro de este archivo se configura el tema visual, se crea el controlador de navegación y se llama a la función Navegacion.

También se carga la preferencia de modo oscuro mediante ThemePreferences y ThemeState.

Otra parte importante es que, al destruirse la actividad, se llama a:

`MusicManager.pararMusica()`

Esto permite liberar la música y evitar que siga sonando en segundo plano cuando se cierra la app.

En resumen, `MainActivity.kt` se encarga de:

- Arrancar la aplicación.
- Activar el diseño de pantalla completa.
- Cargar el tema visual.
- Crear la navegación.
- Detener la música al cerrar la app.

4. Navegacion.kt

`Navegacion.kt` define las pantallas disponibles de la aplicación y las rutas que permiten moverse entre ellas.

La navegación se realiza con `NavHost` y `NavHostController`.

Algunas rutas definidas son:

`inicio`

`inicioSesionCasual`

`inicioSesionCompeti`

`Ranking`

`RegistroCasual`

`RegistroCompeti`

`Juego`

`cambiarNombre`

`gacha`

`coleccion`

`terminos`

Cada ruta está asociada a una pantalla concreta. Por ejemplo:

`inicio` → `PantallaInicio`

`Juego` → `Juego`

`gacha` → `PantallaGacha`

`coleccion` → `PantallaColeccion`

Ranking → Ranking

Este archivo es importante porque centraliza el flujo de la app. Si se añade una pantalla nueva, debe registrarse aquí para que pueda abrirse desde el resto de la aplicación.

5. PantallaInicial.kt

PantallaInicial.kt contiene la pantalla principal de la app.

Es la primera pantalla que ve el usuario al abrir la aplicación. Incluye:

- Logo de Genshiken.
- Animaciones suaves.
- Botones para modo casual y modo competitivo.
- Acceso al ranking.
- Acceso al gacha.
- Acceso a la colección.
- Curiosidad aleatoria.
- Menú de configuración.
- Música de fondo del menú.

La pantalla inicial también utiliza MusicManager para reproducir la música del menú.

Desde esta pantalla el usuario puede elegir cómo quiere usar la app:

- Entrar en modo casual.
- Entrar en modo competitivo.
- Consultar ranking.
- Ir al gacha.
- Ver colección.
- Cambiar nombre.
- Activar o desactivar modo oscuro.

Visualmente es una de las pantallas más importantes, porque sirve como menú principal y presentación del proyecto.

6. Inicio de sesión y registro

La app tiene pantallas separadas para registro e inicio de sesión en modo casual y competitivo.

Los archivos principales son:

InicioSesionCasual.kt

InicioSesionCompetitivo.kt

RegistroCasual.kt

RegistroCompeti.kt

6.1. InicioSesionCompetitivo.kt

InicioSesionCompetitivo.kt es una de las pantallas más importantes, porque es la pantalla usada para entrar al modo competitivo.

Desde esta pantalla el usuario introduce:

- Correo electrónico.
- Contraseña.

Cuando pulsa el botón de entrar, se llama a `UserRepository.login()`. Si el login es correcto, se guarda el usuario en `UserSession`, se cargan monedas, se cargan espadas desbloqueadas y se navega a la pantalla de juego.

Además, en esta pantalla se añadió la llamada para registrar instalaciones de la app:

```
UserRepository.registrarDescarga(  
    usuarioId = id,  
    nombreUsuario = nombre  
)
```

Esto permite que, cuando el usuario inicia sesión correctamente en modo competitivo, se registre una instalación o primer uso en el panel de administración.

La instalación solo se registra una vez para el mismo usuario, dispositivo y versión de app. Esto evita duplicados si el usuario cierra sesión y vuelve a entrar.

6.2. InicioSesionCasual.kt

InicioSesionCasual.kt permite acceder al modo casual.

El modo casual está pensado como una forma más sencilla o de prueba para entrar a jugar. Aunque también puede usar formularios de acceso, el modo importante para ranking, instalaciones y uso completo del sistema es el modo competitivo.

En la documentación del proyecto se considera que el registro de instalaciones se realiza desde el modo competitivo, porque es el modo principal de uso real de la app.

6.3. RegistroCompeti.kt

RegistroCompeti.kt permite crear una cuenta para el modo competitivo.

Desde esta pantalla el usuario introduce:

- Nombre de usuario.
- Correo electrónico.
- Contraseña.
- Confirmación o aceptación de condiciones si corresponde.

El registro llama a la API correspondiente mediante `UserRepository.register()`. Si el registro se realiza correctamente, el usuario queda guardado en la base de datos, dentro de la tabla usuarios.

Esta pantalla está relacionada con la sección de **Usuarios registrados** del panel de administración.

6.4. RegistroCasual.kt

RegistroCasual.kt cumple una función similar, pero para el modo casual.

Sirve como pantalla de registro alternativa o de prueba. La parte más importante del proyecto final es el registro competitivo, ya que es el que se relaciona con ranking, puntuaciones e instalaciones.

7. UserSession.kt y GameSession

UserSession.kt contiene objetos que guardan información temporal durante el uso de la app.

El objeto UserSession guarda:

userId

userName

Esto permite saber qué usuario ha iniciado sesión y asociar sus puntuaciones, monedas, espadas e instalaciones a su cuenta.

El objeto GameSession guarda:

lastScore

lastTime

Esto sirve para recordar la última puntuación y el último tiempo, especialmente al mostrar el ranking después de una partida.

Estos datos no se guardan de forma permanente en el dispositivo, sino que se mantienen mientras la app está en ejecución.

8. ApiService.kt

ApiService.kt define todas las llamadas HTTP que puede hacer la app al backend.

Utiliza Retrofit y contiene métodos para comunicarse con archivos PHP del servidor.

Algunas llamadas importantes son:

register.php

login.php

save_score.php

get_ranking.php

change_name.php

guardar_monedas.php

obtener_monedas.php

guardar_espada.php

obtener_espadas.php

resend_verification.php

obtener_preguntas.php

obtener_espadas_gacha.php

registrarDescarga.php

La mayoría de llamadas usan la URL base definida en RetrofitClient.kt, que apunta a:

http://www.shopkatanas.com/WEB_genshi/api_genshiken/

Sin embargo, la llamada de instalaciones usa una URL completa porque el archivo está en la API normal:

http://www.shopkatanas.com/WEB_genshi/api/registrarDescarga.php

Este archivo es fundamental porque es el puente entre Android y las APIs PHP.

9. RetrofitClient.kt

RetrofitClient.kt configura Retrofit para poder realizar llamadas al backend.

Contiene la URL base de las APIs principales:

http://www.shopkatanas.com/WEB_genshi/api_genshiken/

Además, añade el conversor Gson para poder convertir respuestas JSON en objetos Kotlin.

Su función es crear una instancia única de ApiService, que después se utiliza desde UserRepository.

En resumen, RetrofitClient.kt se encarga de:

- Definir la URL base.
- Configurar Retrofit.
- Crear el servicio API.
- Permitir que el resto de la app haga peticiones al servidor.

10. UserRepository.kt

UserRepository.kt centraliza la comunicación entre la app y el backend.

En vez de hacer llamadas Retrofit directamente desde cada pantalla, la app utiliza métodos de UserRepository.

Este archivo contiene funciones como:

- login()
- register()
- registrarDescarga()
- saveScore()
- getRanking()
- changeName()
- guardarMonedas()
- obtenerMonedas()
- guardarEspada()
- obtenerEspadas()
- reenviarVerificacion()
- obtenerPreguntas()
- obtenerEspadasGacha()

Esto mejora la organización del código, ya que las pantallas solo llaman a funciones simples y no tienen que preocuparse por los detalles de Retrofit.

Por ejemplo, la pantalla de login competitivo llama a UserRepository.login(), y después, si el login es correcto, llama a UserRepository.registrarDescarga().

También se encarga de obtener el nombre del dispositivo mediante datos del sistema Android, usando información como fabricante y modelo.

11. Juego.kt

Juego.kt contiene la lógica principal del juego de preguntas.

Esta pantalla se encarga de:

- Cargar preguntas desde la API.
- Mostrar la pregunta actual.
- Mostrar la imagen de la pregunta.
- Mostrar las respuestas disponibles.
- Controlar si el usuario acierta o falla.
- Cambiar de pregunta.
- Controlar la puntuación.
- Controlar el tiempo.
- Guardar la puntuación al finalizar.
- Reproducir la música de juego.
- Gestionar el avance entre niveles.

Las preguntas se obtienen desde el backend mediante:

`UserRepository.obtenerPreguntas(nivel)`

Cada pregunta tiene:

- ID.
- Texto.
- Imagen.
- Lista de opciones.
- Índice de la opción correcta.

Durante la partida se muestra una barra o progreso de preguntas, y cuando el jugador responde se indica visualmente si la respuesta es correcta o incorrecta.

Al terminar el nivel, la app puede guardar la puntuación y sumar monedas al usuario. Estas monedas luego se pueden usar en el gacha.

12. Preguntas.kt

Preguntas.kt define el modelo de datos de una pregunta.

La clase contiene campos como:

`id`

`pregunta`

`imagenPregunta`

`opciones`

`opcionCorrecta`

Esta clase permite que Retrofit convierta la respuesta JSON de la API en objetos Kotlin que después se muestran en la pantalla de juego.

13. EstadoRespuesta.kt

`EstadoRespuesta.kt` define los posibles estados visuales de una respuesta durante el juego.

Sirve para saber si una opción está:

- Sin seleccionar.
- Seleccionada.
- Correcta.
- Incorrecta.

Esto ayuda a pintar las respuestas con colores diferentes y dar feedback visual al usuario.

14. Ranking.kt

`Ranking.kt` muestra el ranking de jugadores.

La pantalla obtiene los datos mediante:

`UserRepository.getRanking()`

El ranking permite seleccionar mes y año, de forma que se puede consultar el ranking de diferentes periodos.

Los datos se representan mediante la clase `Player`, definida en `Jugador.kt`.

El ranking muestra información como:

- Posición.
- Nombre.
- Puntuación.
- Tiempo.

Esta pantalla es importante para el modo competitivo, ya que permite comparar los resultados entre jugadores.

15. Jugador.kt

Jugador.kt contiene la clase Player.

Esta clase representa a un jugador dentro del ranking.

Sus campos son:

posicion

usuarioId

nombre

puntuacion

tiempo

Gracias a esta clase, la app puede recibir el ranking desde la API y mostrarlo de forma ordenada.

16. Sistema de gacha

El sistema de gacha permite que el usuario utilice monedas para desbloquear espadas aleatorias.

Las clases principales del sistema son:

PantallaGacha.kt

PantallaColeccion.kt

GachaState.kt

EspadaOnline.kt

EspadaGacha.kt

16.1. PantallaGacha.kt

PantallaGacha.kt muestra la pantalla donde el usuario puede tirar el gacha.

Desde esta pantalla el usuario puede gastar monedas para intentar conseguir una espada.

El sistema comprueba si el usuario tiene monedas suficientes. Si tiene monedas, se realiza una tirada aleatoria y se obtiene una espada.

Después se guarda:

- La nueva cantidad de monedas.
- La espada desbloqueada.
- La colección del usuario.

El gacha utiliza rarezas como:

COMUN

RARA

EPICA

LEGENDARIA

Cada rareza tiene distinta probabilidad de aparecer.

16.2. EspadaOnline.kt

EspadaOnline.kt define el modelo de una espada obtenida desde la base de datos.

Sus campos principales son:

id

nombre

rareza

descripcion

imagen_url

Esta clase representa las espadas que se gestionan desde el panel de administración y se reciben desde la API.

También incluye la función tirarGacha(), que selecciona una espada aleatoria en función de la rareza.

Las probabilidades usadas son aproximadamente:

COMUN → 60%

RARA → 30%

EPICA → 9%

LEGENDARIA → 1%

Si no hay espadas de una rareza concreta, la función puede devolver una espada aleatoria de la lista disponible.

16.3. GachaState.kt

GachaState.kt guarda el estado del sistema de gacha mientras la app está abierta.

Contiene:

monedas

espadasDesbloqueadas

listaEspadasOnline

Esto permite saber:

- Cuántas monedas tiene el usuario.
- Qué espadas ha desbloqueado.
- Qué espadas existen en el catálogo online.

También tiene una función para añadir espadas evitando duplicados.

16.4. PantallaColeccion.kt

PantallaColeccion.kt muestra la colección de espadas del usuario.

Desde esta pantalla se ven:

- Espadas desbloqueadas.
- Espadas bloqueadas.
- Imagen de cada espada.
- Nombre.
- Rareza.
- Descripción.
- Progreso de colección.
- Enlaces clicables dentro de la descripción.

Una mejora importante realizada fue permitir que los enlaces escritos en la descripción de una espada fueran clicables también en Android. Por ejemplo, si la descripción contiene:

`http://www.google.com`

la app lo muestra como enlace y permite abrirlo en el navegador del móvil.

Esto demuestra que el enlace se guarda correctamente desde el panel, llega a la app y funciona en Android.

16.5. EspadaGacha.kt

EspadaGacha.kt contiene una estructura de espadas locales o de apoyo.

En versiones iniciales del proyecto se usaban espadas definidas directamente en Android. Después se mejoró el sistema para poder obtener espadas desde la base de datos mediante EspadaOnline.kt y obtener_espadas_gacha.php.

Este archivo sirve como referencia o apoyo para el sistema de espadas, pero la versión más dinámica del proyecto utiliza las espadas online gestionadas desde el panel.

17. Música y sonidos

La app incluye música y sonidos para mejorar la experiencia de usuario.

Los archivos de sonido se guardan en:

`app/src/main/res/raw/`

Entre ellos están:

`musica_menu.ogg`

`musica_juego.ogg`

`error.mp3`

`katanasonido.mp3`

17.1. MusicManager.kt

MusicManager.kt controla la música de la app.

Sus funciones principales son:

`reproducirMenu()`

`reproducirJuego()`

`pararMusica()`

La música del menú se reproduce en la pantalla inicial, mientras que la música del juego se reproduce durante la partida.

El objetivo de esta clase es evitar que se solapen varias canciones y liberar recursos cuando la app se cierra.

18. Tema visual y preferencias

La app incluye soporte para modo oscuro y configuración visual.

Los archivos relacionados son:

ThemeState.kt

ThemePreferences.kt

ui/theme/Color.kt

ui/theme/Theme.kt

ui/theme/Type.kt

18.1. ThemeState.kt

ThemeState.kt guarda el estado actual del tema, por ejemplo si el modo oscuro está activado o no.

18.2. ThemePreferences.kt

ThemePreferences.kt permite guardar y cargar la preferencia de modo oscuro usando almacenamiento local.

Así, si el usuario cambia el modo oscuro, la app puede recordar esa preferencia.

19. Cambio de nombre

El archivo:

CambiarNombre.kt

permite al usuario modificar su nombre.

La app llama a la API correspondiente mediante:

UserRepository.changeName()

El usuario debe introducir los datos necesarios y, si son correctos, el nombre se actualiza en la base de datos.

20. Términos de servicio

El archivo:

TerminosServicios.kt

muestra una pantalla de términos o condiciones de uso.

Esta pantalla está relacionada con el proceso de registro, ya que permite mostrar información legal o de condiciones antes de crear una cuenta.

21. Curiosidades

Curiosidades.kt contiene una lista de frases o curiosidades que se muestran en la pantalla inicial.

La pantalla inicial selecciona una curiosidad aleatoria para hacer el menú más dinámico y atractivo.

22. Base de datos

La base de datos utilizada en el proyecto es MySQL. Es compartida por la app Android y el panel de administración.

Las tablas principales son:

usuarios
preguntas
respuestas
niveles
puntuaciones
ranking_mensual_historico
descargas
espadas
monedas_usuario
coleccion_usuario

También existen tablas adicionales como:

dispositivos
intentos
intento_preguntas
modos
ganadores_mensuales
espadas_gacha

22.1. Tabla usuarios

La tabla usuarios almacena las cuentas registradas en la app.

Guarda datos como:

- ID del usuario.
- Nombre.
- Correo electrónico.
- Contraseña o hash.
- Estado de verificación.
- Fecha de registro.

Esta tabla se muestra en el panel dentro de **Usuarios registrados**.

22.2. Tabla preguntas

La tabla preguntas almacena las preguntas del juego.

Cada pregunta tiene:

- ID.
- Texto.
- Imagen.
- Nivel asociado.

Gracias al campo de nivel, el juego puede separar las preguntas en nivel 1, nivel 2 y futuros niveles.

22.3. Tabla respuestas

La tabla respuestas almacena las respuestas posibles para cada pregunta.

Cada respuesta está asociada a una pregunta mediante pregunta_id.

También indica si una respuesta es correcta o no.

22.4. Tabla niveles

La tabla niveles almacena los niveles del juego.

Permite organizar el contenido y cargar preguntas según el nivel seleccionado en la app.

22.5. Tabla puntuaciones

La tabla puntuaciones guarda las puntuaciones obtenidas por los usuarios.

Puede almacenar:

- Usuario.
- Nombre.
- Puntuación.
- Tiempo.
- Fecha.

Esta tabla se usa para construir el ranking.

22.6. Tabla ranking_mensual_historico

La tabla ranking_mensual_historico sirve para guardar información de rankings anteriores.

Es útil cuando se reinicia el ranking mensual y se quiere conservar un histórico.

22.7. Tabla descargas

La tabla descargas registra instalaciones o primeros usos de la app.

Guarda datos como:

- Usuario.
- Nombre de usuario.
- Dispositivo.
- Versión de la app.
- Fecha.

En el proyecto se decidió registrar una instalación cuando el usuario inicia sesión correctamente en modo competitivo.

También se evita registrar duplicados del mismo usuario, dispositivo y versión.

22.8. Tabla espadas

La tabla espadas almacena el catálogo de espadas del gacha.

Cada espada tiene:

- ID.
- Nombre.
- Rareza.
- Descripción.
- URL de imagen.

Esta tabla se gestiona desde el panel de administración y la app la consulta para mostrar el gacha y la colección.

22.9. Tabla monedas_usuario

La tabla monedas_usuario guarda las monedas que tiene cada usuario.

Estas monedas se pueden ganar jugando y gastar en el gacha.

22.10. Tabla coleccion_usuario

La tabla coleccion_usuario guarda las espadas desbloqueadas por cada usuario.

Relaciona:

- Usuario.
- Espada.

Esto permite que cada jugador tenga su propia colección.

23. Manual básico de uso de la app

23.1. Abrir la app

Al abrir la app aparece la pantalla inicial con el logo de Genshiken, música de menú y accesos a las secciones principales.

23.2. Registrarse

El usuario puede registrarse desde la pantalla de registro competitivo.

Debe introducir sus datos y completar el proceso. Después aparecerá en la sección de usuarios registrados del panel.

23.3. Iniciar sesión en competitivo

El usuario entra en modo competitivo e introduce correo y contraseña.

Si los datos son correctos:

- Se guarda la sesión.
- Se cargan monedas.
- Se carga la colección.
- Se registra la instalación.
- Se entra al juego.

23.4. Jugar

Durante la partida el usuario responde preguntas con varias opciones.

El sistema indica si la respuesta es correcta o incorrecta. Al terminar se guarda la puntuación y se actualiza el ranking.

23.5. Consultar ranking

Desde el menú se puede acceder al ranking.

El usuario puede ver los mejores resultados del mes y año seleccionados.

23.6. Usar el gacha

Desde la pantalla de gacha el usuario puede gastar monedas para obtener espadas aleatorias.

Las espadas se guardan en su colección.

23.7. Ver colección

En la colección se muestran las espadas obtenidas y las bloqueadas.

Si una espada tiene un enlace en la descripción, el usuario puede pulsarlo y abrirlo en el navegador.

24. Manual básico de uso del panel

El administrador puede entrar al panel desde:

http://www.shopkatanas.com/WEB_genshi/admin/login.php

Desde ahí puede:

- Gestionar preguntas.
- Añadir preguntas de nivel 1 y nivel 2.
- Subir imágenes.
- Editar respuestas.
- Revisar ranking.
- Ver usuarios registrados.
- Ver instalaciones.
- Gestionar espadas del gacha.
- Añadir enlaces en descripciones.

El panel es la herramienta principal para mantener el contenido sin tocar código.

25. Pruebas realizadas

Durante las prácticas se realizaron pruebas tanto en Android como en la web.

25.1. Pruebas de Android

Se probaron:

- Pantalla inicial.
- Música del menú.
- Música del juego.
- Registro de usuario.
- Login competitivo.
- Carga de preguntas.
- Nivel 1.
- Nivel 2.
- Respuestas correctas e incorrectas.
- Guardado de puntuación.
- Ranking.
- Monedas.
- Gacha.
- Colección.
- Enlaces clicables.
- Registro de instalaciones.

25.2. Pruebas de web

Se probaron:

- Login admin.
- Dashboard.
- Gestión de preguntas.
- Selector de imágenes.
- Gestión de respuestas.
- Ranking.
- Usuarios registrados.
- Instalaciones.
- Gestión de espadas.
- Enlaces clicables en descripciones.
- Subida de imágenes del gacha.

25.3. Pruebas de API

Se probaron APIs desde:

- La app Android.
- El navegador.
- PowerShell.
- phpMyAdmin.

Una prueba importante fue la de registrarDescarga.php, que confirmó que el hosting podía registrar instalaciones correctamente.

26. Mejoras realizadas en la app durante las prácticas

Las principales mejoras realizadas en la app fueron:

- Mejora visual de la pantalla inicial.
- Añadida música de menú.
- Añadida música de juego.
- Conexión con preguntas desde backend.
- Integración con ranking.
- Sistema de gacha conectado con base de datos.
- Sistema de colección de espadas.
- Carga de imágenes desde URL.
- Enlaces clicables en la colección.
- Registro de instalaciones desde login competitivo.
- Soporte para nivel 2.
- Limpieza de navegación.
- Mejora de experiencia visual con tarjetas, colores y animaciones.

27. Mejoras realizadas en la base de datos

Durante las prácticas también se mejoró la base de datos para soportar nuevas funcionalidades:

- Registro de usuarios.
- Gestión de preguntas por niveles.
- Respuestas asociadas a preguntas.
- Ranking mensual.
- Historial de ranking.
- Registro de instalaciones.
- Espadas del gacha.
- Monedas por usuario.
- Colección de espadas por usuario.

La base de datos quedó preparada para ampliar el proyecto con más niveles, más preguntas y más espadas.

28. Posibles mejoras futuras

Aunque el proyecto ya es funcional, se podrían añadir mejoras futuras:

- Añadir más niveles.
- Añadir más preguntas.
- Añadir más espadas al gacha.
- Añadir notificaciones.
- Añadir estadísticas personales.
- Publicar la app en una tienda oficial.
- Mejorar el diseño responsive de la web.
- Añadir logs de errores para mantenimiento.
- Añadir sistema de temporadas o eventos.

Estas mejoras permitirían que el proyecto siguiera creciendo después de las prácticas.

29. Conclusión final

El proyecto **Trivia GenshikenC.S.** terminó siendo un sistema completo formado por una aplicación Android, un panel web de administración, APIs PHP y una base de datos MySQL.

La app permite al usuario registrarse, iniciar sesión, jugar preguntas por niveles, guardar puntuaciones, consultar el ranking, conseguir monedas, usar el gacha y ver su colección de espadas.

El panel permite al administrador gestionar preguntas, respuestas, imágenes, ranking, usuarios, instalaciones y espadas del gacha sin modificar directamente el código.

La base de datos centraliza toda la información y permite que la app y el panel trabajen conectados.

Durante las prácticas se realizaron mejoras importantes tanto en la parte Android como en la web. Entre las más destacadas están el selector de imágenes, la gestión de espadas del gacha, los enlaces clicables, la música de la app, el registro de instalaciones y la creación del nivel 2.

En conclusión, el proyecto cumple su objetivo principal: ofrecer una aplicación de trivia funcional, conectada a un backend real y administrable desde una web. Además, queda preparado para poder ampliarse en el futuro con nuevos niveles, nuevas preguntas, más espadas y nuevas funcionalidades.